

COMPUTACIÓN DE ALTAS PRESTACIONES

PRÁCTICA 2: Programación de GPU

AUTORES

CARLOS GARCÍA SANTA

CELIA MARTÍN ÁLVAREZ

GRUPO 1461 : PAREJA 03

INGENIERÍA INFORMÁTICA

ESCUELA POLITÉCNICA SUPERIOR

UNIVERSIDAD AUTÓNOMA DE MADRID



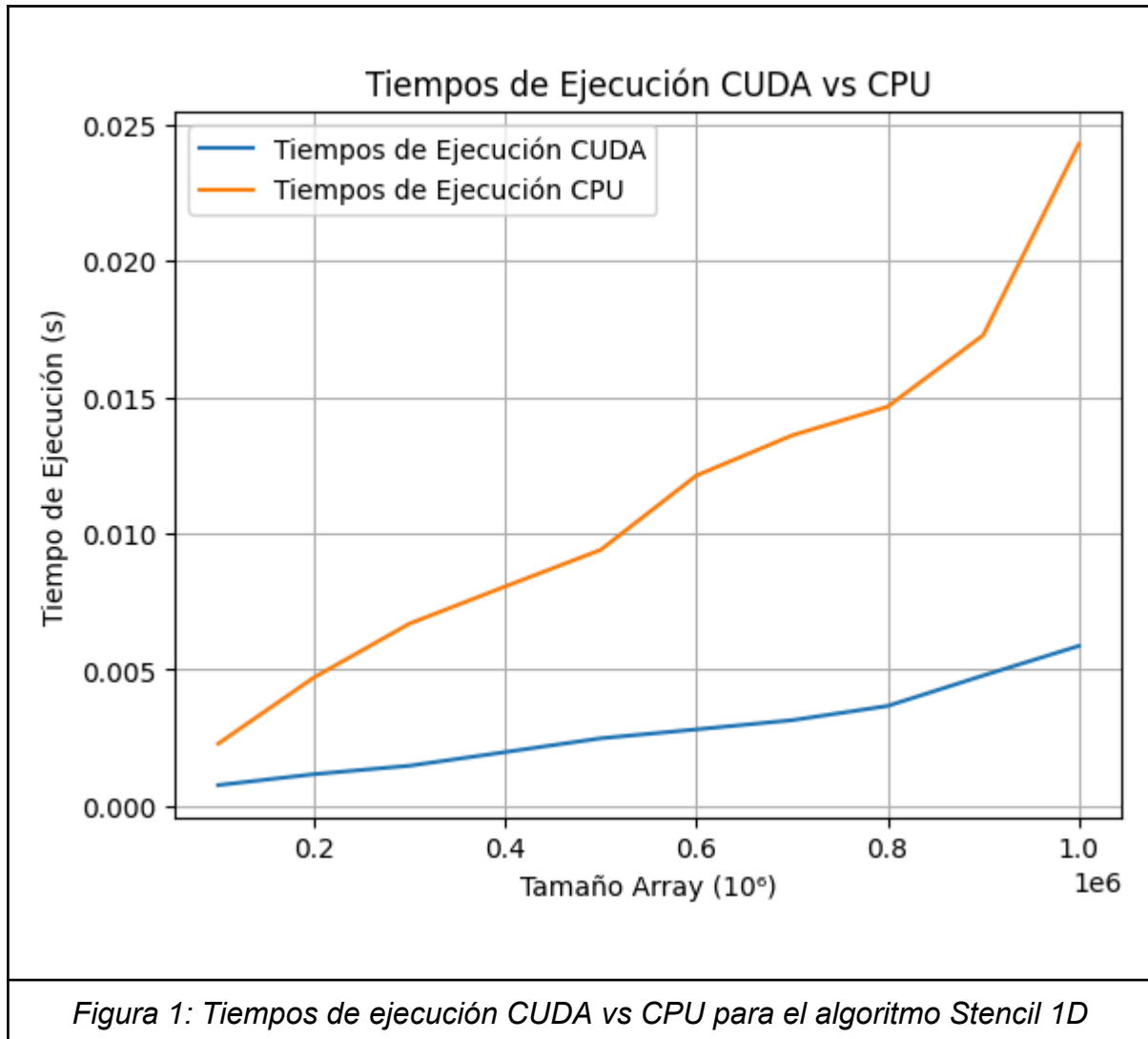
28/10/2024

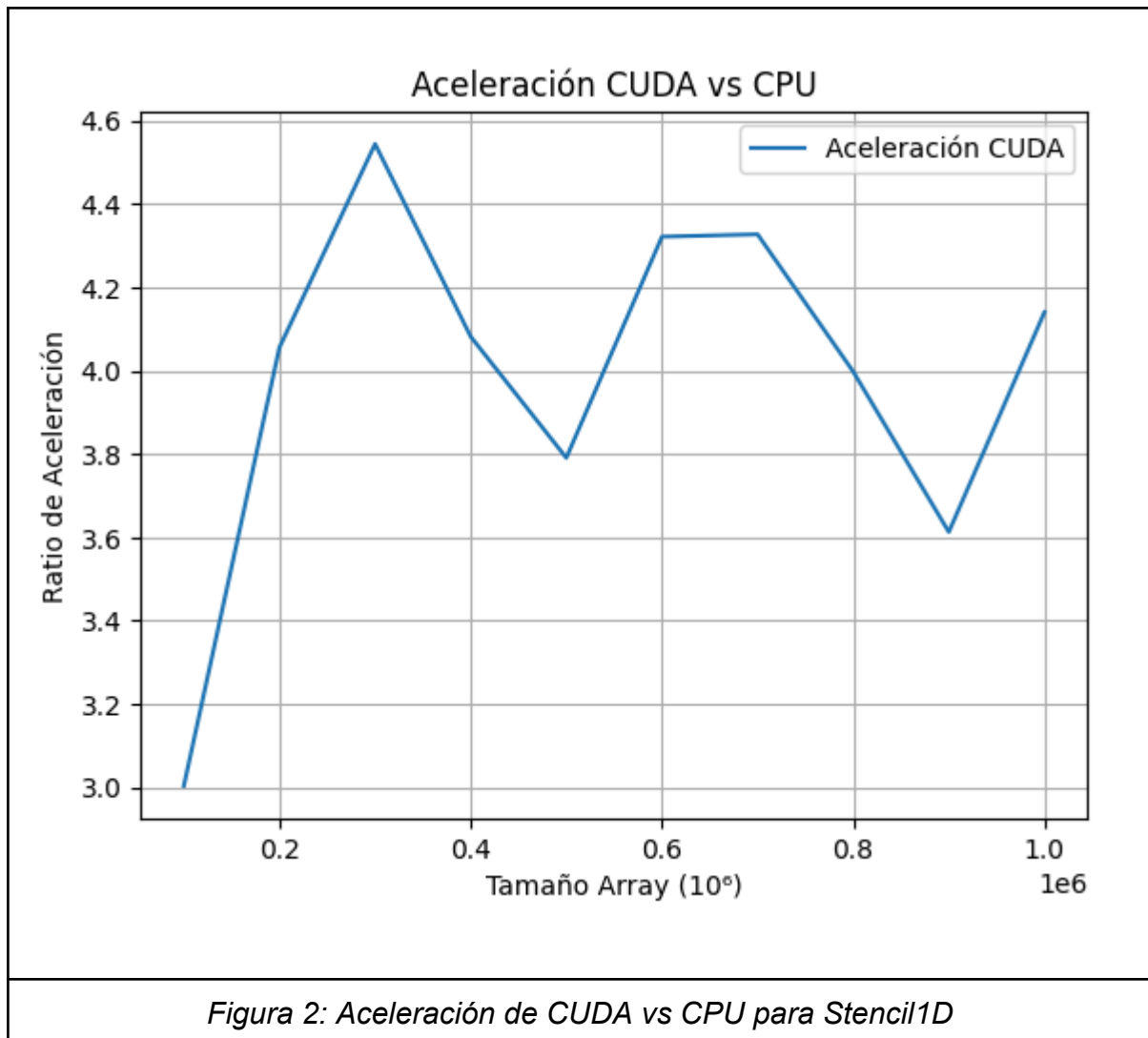
Índice

Ejercicio 1.....	2
Ejercicio 2.....	5
Ejercicio 3.....	8

EJERCICIO 1: Stencil 1D

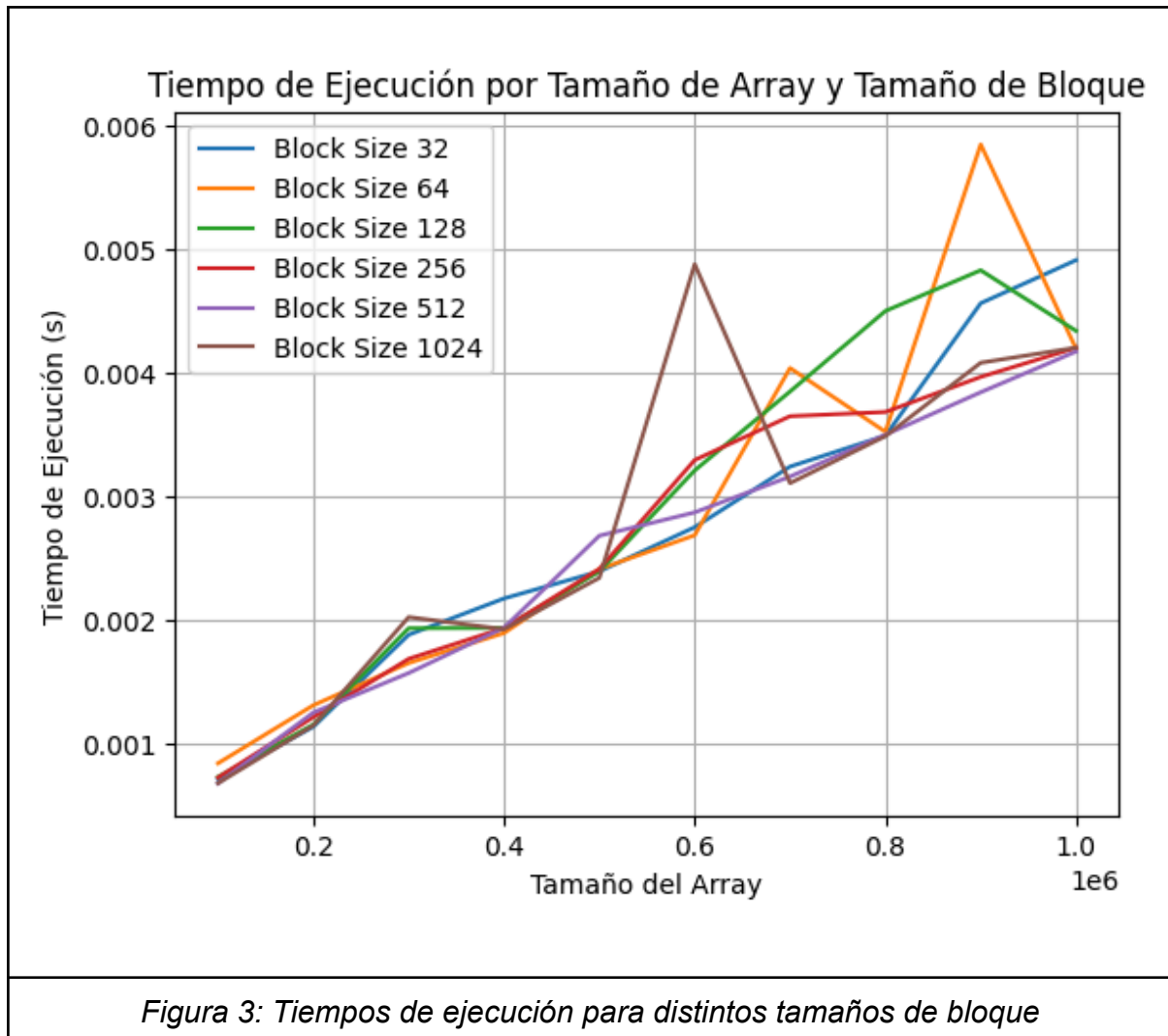
2. Compare el tiempo de ejecución para diferentes valores de N (tamaño de matriz): de 100.000 a 1.000.000 en pasos de 100.000. Represente el resultado en una gráfica. Explique los resultados.





Los tiempos de ejecución son significativamente menores con la GPU, teniendo una curva de tiempos de ejecución mucho más pronunciada para la CPU, lo que resulta en una aceleración considerable, que varía entre un rango de 3.0 y 4.5 para los tamaños de arrays elegidos. Esto nos indica que se está aprovechando de manera efectiva la paralelización que ofrece la GPU mediante CUDA para el algoritmo Stencil1D. Sin embargo, es importante destacar las fluctuaciones que se observan en la aceleración, las cuales pueden darse por el coste de transferencia de datos entre el host (CPU) y el dispositivo (GPU), que se vuelven más relevantes a medida que aumenta el tamaño del array, a pesar de ello, el uso de CUDA sigue siendo claramente mejor, manteniendo los tiempos de ejecución mucho más bajos que los de la CPU, especialmente cuando hay mayor carga computacional.

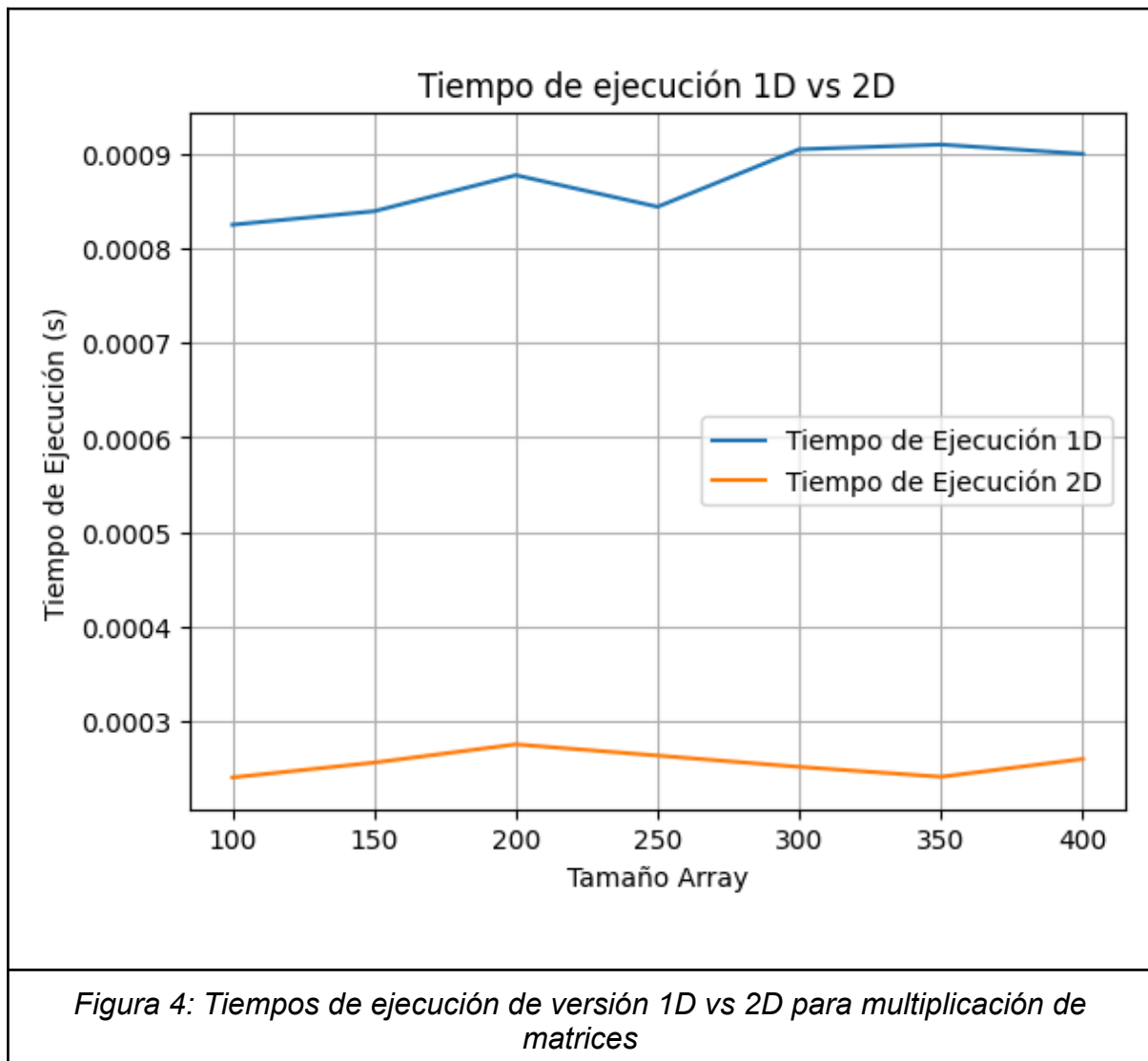
3. ¿Qué TAMAÑO DE BLOQUE (número de hilos por bloque) has utilizado?
¿Crees que es lo óptimo? Explique los resultados.



Para los tamaños de array elegidos no se observan diferencias significativas en los tiempos de ejecución en función del tamaño del bloque. Se han elegido múltiplos de 32 para el número de hilos por bloque, dado que los warps en CUDA están conformados por conjuntos de 32 hilos, de esta forma evitamos que algunos hilos queden inactivos y se pierda eficiencia. Por este motivo se ha elegido como tamaño de bloque 256, puesto que es un múltiplo de 32 y podemos disponer de 8 warps ejecutándose cada uno en un multiprocesador, aunque es importante destacar que experimentalmente no observamos ningún cambio de tiempos de ejecución entre cualquier múltiplo de 32.

EJERCICIO 2: Tutorial multiplicación de matrices con GPU

2.1. En el primer ejemplo del tutorial se utiliza un grid de bloques 2D y en cada bloque una distribución de hilos también 2D. Realice una versión 1D tanto para el lanzamiento de bloques, como para la distribución de hilos dentro del bloque. ¿Qué versión es más eficiente? Justifique los resultados.

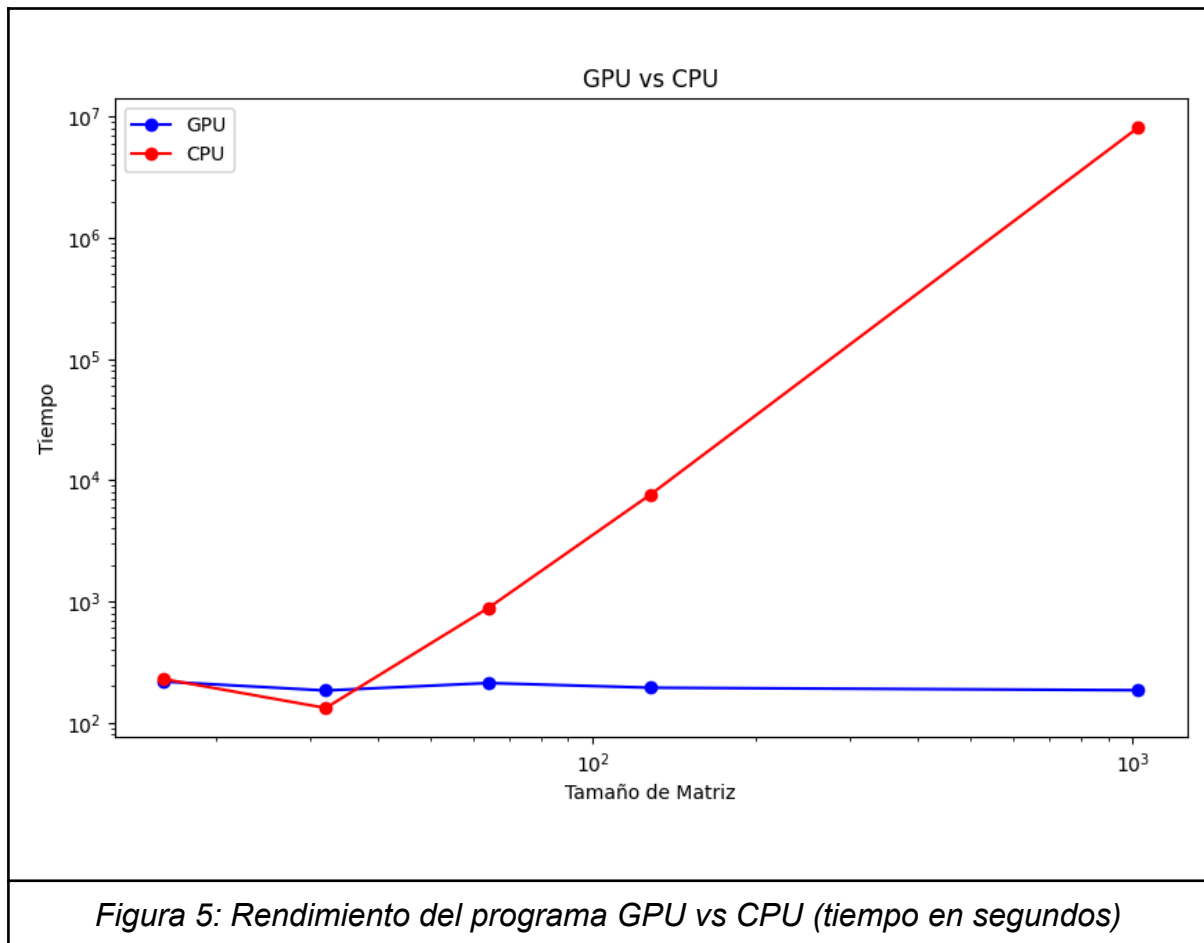


Al comparar el rendimiento entre la multiplicación de matrices en 1D y en 2D podemos observar que al hacerlo en 2D el porcentaje de tiempo de las actividades en la GPU al realizar la multiplicación de matrices ocupa menos porcentaje de tiempo que al hacerlo en 1D.

Como reflejamos en la gráfica vemos que al hacer la media de las cinco iteraciones del bucle, el resultado es que las matrices en una dimensión tardan más que al

hacerlo en dos dimensiones. Esto puede deberse a que en la memoria los datos están organizados en filas y columnas que es de la forma en la que están compuestas las matrices 2D.

2.2. Para el segundo ejemplo del tutorial estudie el rendimiento del programa en función de N. Para guiarle en este estudio, se plantean a continuación diferentes cuestiones a responder:



• ¿Qué valor de N obtiene $T_{gpu}=T_{cpu}$? ¿Qué pasa antes y después de este N?
¿Por qué?

Para que el tiempo de CPU y el tiempo de GPU coincidan tienen que tener un tamaño de matriz pequeño, inferior a 10^2 . Como podemos observar en la gráfica cuando la matriz es más pequeña que 10^2 hay un punto en el tamaño que se cruzan y coinciden la CPU y la GPU.

Cuando la matriz es pequeña el tiempo de GPU es mayor que el de CPU, esto es debido a que se sobrecarga el número de hilos que se lanzan para poder calcular matrices pequeñas. Mientras que para tamaños grandes el tiempo de CPU es mayor

que el de GPU, ya que la GPU puede lanzar los hilos sin sobrecargarse y poder paralelizar las operaciones para que el rendimiento sea más efectivo.

• **¿Cuál es el máximo valor de N que logra ejecutarse en la GPU? ¿Por qué?**

Al ejecutar nvidia-smi vemos que tenemos 15GB de memoria disponible de GPU. El programa usa tres matrices: las matrices a y b que son las que se van a multiplicar y luego la matriz c que es la matriz resultante. Todas las matrices son de tamaño $n * n$. Por lo que para calcular la memoria total utilizada sería multiplicar el tamaño $n * n$ por 3, ya que utilizamos tres matrices y a su vez multiplicarlo por el tamaño del tipo de dato utilizado. En este caso sería: $3 * n * n * 4$ bytes (float). Como n es el tamaño de la matriz, vamos probando con distintos tamaños.

- $(3 * 16 * 16 * 4) / (1024 * 1024) = 0,00029\text{MB}$
- $(3 * 32 * 32 * 4) / (1024 * 1024) = 0,01172\text{MB}$
- $(3 * 64 * 64 * 4) / (1024 * 1024) = 0,04688\text{MB}$
- $(3 * 128 * 128 * 4) / (1024 * 1024) = 0,1875\text{MB}$
- $(3 * 1024 * 1024 * 4) / (1024 * 1024) = 12\text{MB}$
- $(3 * 2048 * 2048 * 4) / (1024 * 1024) = 48\text{MB}$

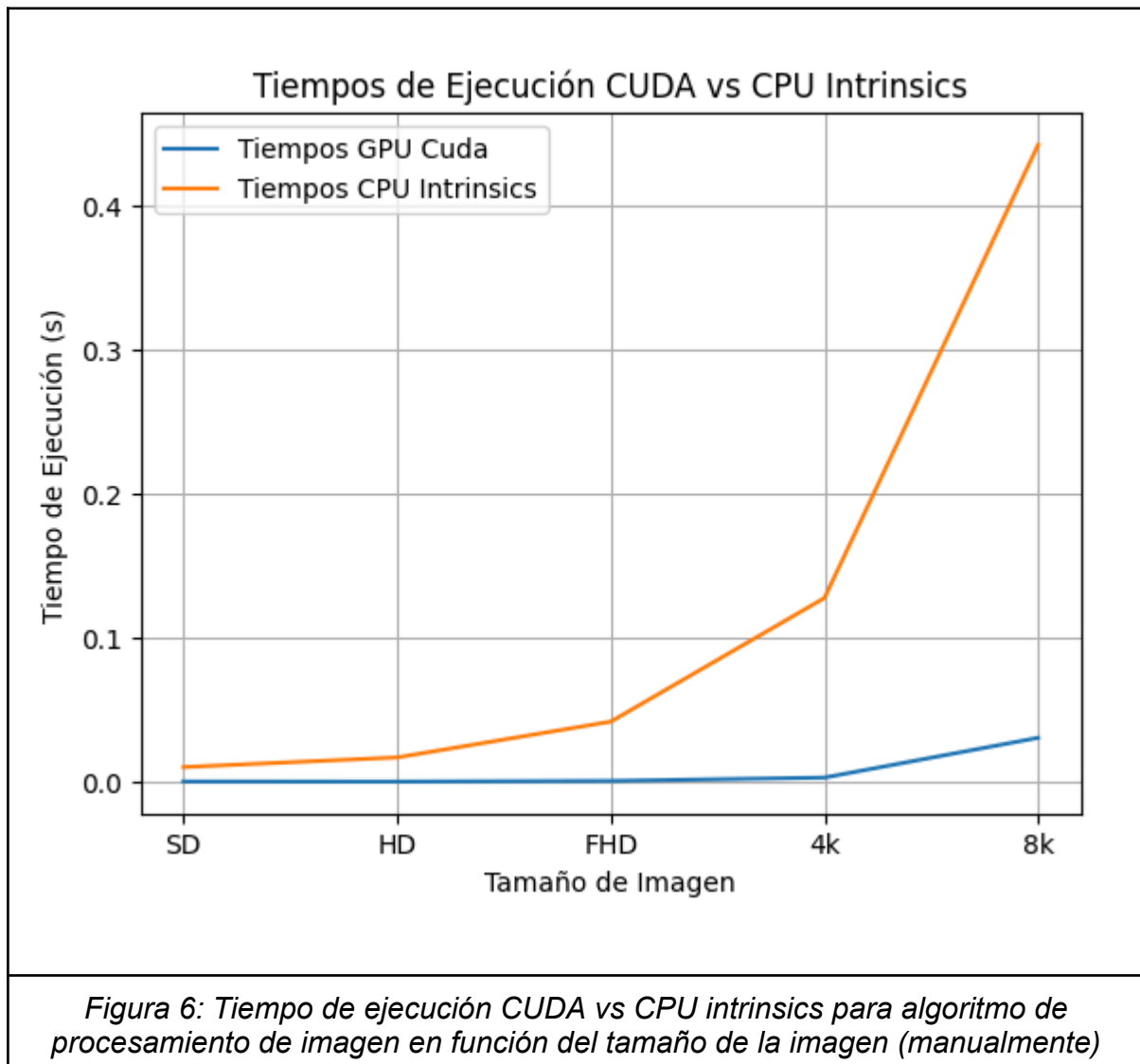
Como podemos ver con tamaño $N = 2048$ se pasa de la memoria disponible de la GPU, por lo que el máximo valor sería para $N = 1024$.

• **¿Qué coste extra en las operaciones de memoria tiene la GPU frente a la CPU? ¿Qué consideraría “justo” añadir para comparar con la CPU?**

La GPU tiene un coste extra en las operaciones frente a CPU, debido a las transferencias de memoria entre el host y la GPU. Antes de hacer cualquier operación hacemos esto `cudaMemcpyHostToDevice` para copiar lo que tenemos en el host en la memoria de la GPU. Otra de las cosas que hace la GPU a diferencia de la CPU, es el lanzamiento de hilos y su inicialización.

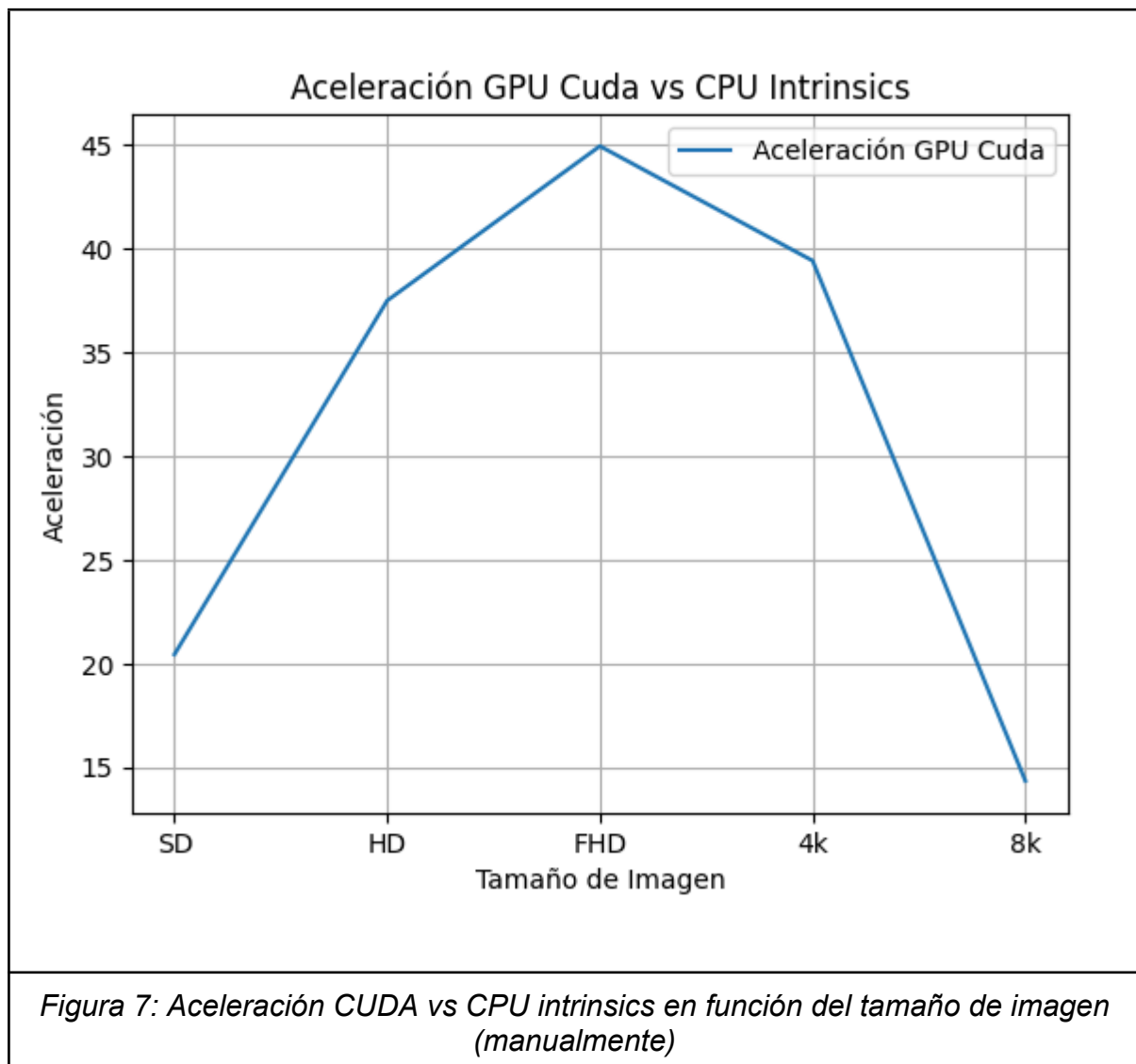
EJERCICIO 3: Procesamiento de imagen

2. Complete una tabla con los resultados de tiempo y velocidad en comparación con el código de su CPU vectorizado manualmente para imágenes de diferentes resoluciones (SD, HD, FHD, UHD-4k, UHD-8k). Debe incluir una columna con los fps a los que procesaría el programa. Discuta los resultados.



Los resultados de los tiempos de ejecución muestran claramente que la implementación en GPU mediante CUDA es considerablemente más eficiente que la implementada con intrinsic (implementados de forma manual) de CPU para el procesamiento del algoritmo de paso de imágenes rgb a escala de grises. En la gráfica observamos como los tiempos de ejecución en GPU se mantienen bajos y casi constantes, incluso para resoluciones altas como 4K y 8K, mientras que en

CPU los tiempos crecen exponencialmente con el tamaño de la imagen. Esta mejora se debe a que las imágenes son objetos que contienen una gran cantidad de datos que se pueden procesar en paralelo, en este caso, con la GPU podemos procesar cada píxel de forma paralela empleando los múltiples hilos paralelos que pueden ejecutar los multiprocesadores, mientras que con la CPU la cantidad de hilos en paralelo que se pueden ejecutar es limitada, por ello la GPU es considerablemente mejor en esta tarea.



La aceleración va de un factor de 15 a 45 siendo máxima en imágenes FHD, donde la GPU es 45 veces más rápida que la CPU, pero disminuye de forma considerable en 4k y 8K, esto se debe a que cuanto mayor es el tamaño de la imagen mayor es la cantidad de memoria requerida, y por lo tanto los factores asociados a los recursos de memoria como la latencia en la transferencia de datos o la saturación de las

cachés o los registros de los multiprocesadores, limitan considerablemente la aceleración que se puede obtener generando un cuello de botella. A pesar de esto la aceleración sigue siendo muy considerable y refleja la superioridad de la GPU en el procesamiento de imágenes.

Tamaño Imagen	Tiempo GPU (s)	Tiempo CPU (s)	FPS GPU	FPS CPU	Aceleración
SD	0.000516	0.010538	1939.49	94.9	20.44
HD	0.00046	0.017236	2175.81	58.02	37.5
FHD	0.000936	0.042093	1067.92	23.76	44.95
4k	0.003245	0.127943	308.15	7.82	39.43
8k	0.030856	0.442541	32.41	2.26	14.34

Figura 8: Tabla de tiempos, fps y aceleración en función del tamaño de imagen de CUDA vs CPU intrinsics (manualmente)

En la tabla podemos ver que el número de imágenes procesadas por segundo es mucho mayor en la GPU, siendo especialmente notable para las imágenes 4k y 8k, donde la CPU solamente alcanza 7.82 y 2.26 fotogramas por segundo, frente a los 308.15 y 32.41 de la GPU. Esta última medida muestra con claridad que las GPUs son una opción mejor para el procesamiento de imágenes, siendo el rendimiento vectorial con intrinsics de la CPU marginal.